

Reviewer Pack — Minimal Rank of Quasi-Monogenic Sequences Bounds Resolution ...

Entry 5178f9916795 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-27 12:33:08 UTC

Minimal Rank of Quasi-Monogenic Sequences Bounds Resolution Refutation Size

Entry ID: 5178f9916795 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-27 12:33:08 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Number Theory (Quasi-Monogenic Sequences) **Field B** (complexity object): Complexity Theory: Resolution Proof Complexity

Statement:

[‘For a given instance of a Tseitin formula, the minimal rank of its associated quasi-monogenic sequence is at least as large as the resolution refutation size required to satisfy it.’, ‘Formally, for every Tseitin formula F with n variables and m clauses, let $Q(F)$ denote its associated quasi-monogenic sequence. Then, there exists a constant $C > 0$ such that for all instances of F , $\text{rank}(Q(F)) \geq C * \log_2(n + m)$.’, ‘For all instances with $m < n^C$, the associated quasi-monogenic sequence has a minimal rank ≤ 1 .’]

Rationale (proposer’s reasoning):

[‘Quasi-monogenic sequences are algebraic objects that can encode arithmetic properties of sequences. Their ranks have been studied in number theory but rarely applied to complexity theory.’, ‘By linking the rank of these sequences to resolution refutation size, we may uncover a new way to measure complexity and potentially find hard instances for Tseitin formulas.’, ‘This conjecture suggests that arithmetic properties encoded by quasi-monogenic sequences could be a source of hardness in proof complexity.’]

Taxonomy category: TSEITIN_RES (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): e7dc495ad6dd2269

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if all 30 seeds yield $\text{rank}(Q(F)) \geq C * \log_2(n + m)$ for every instance with $m < n^C$, and falsified if any seed shows a $Q(F)$ rank $\leq \bar{C} * \log_2(n + m)$ for an instance with $m < n^C$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	SAFE
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 4 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "quasi-monogenic sequences" AND "resolution refutation size" - "Tseitin formula" AND minimal rank" AND "quasi-monogenic sequence" - "complexity theory" AND resolution AND quasi-monogenic sequences

Top relevant hits considered: - [http://arxiv.org/abs/2103.09609v1] Characterizing Tseitin-formulas with short regular resolution refutations - [http://arxiv.org/abs/2402.00542v1] Hardness of Random Reordered Encodings of Parity for Resolution and CDCL - [http://arxiv.org/abs/2102.05019v2] On the Power and Limitations of Branch and Cut - [http://arxiv.org/abs/2312.12247v2] Bernstein-Gelfand-Gelfand meets geometric complexity theory: resolving the 2 x 2 permanents of a 2 x n matrix

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(A):
    m, n = len(A), len(A[0])
    for i in range(m):
        max_row = i
        for j in range(i + 1, m):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]
        for j in range(i + 1, m):
            factor = Fraction(A[j][i], A[i][i])
            for k in range(n):
                A[j][k] -= factor * A[i][k]
    return A

def rank(matrix):
    matrix = [row[:] for row in matrix]
    r = gaussian_elimination(matrix)
    return sum(1 for row in r if any(row))
```

```

def tseitin_formula(n, m):
    variables = list(range(1, n + 1))
    clauses = []
    for i in range(m):
        clause = [random.choice(variables), random.choice(variables)]
        clauses.append(clause)
    return variables, clauses

def quasi_monogenic_sequence(variables, clauses):
    n = len(variables)
    m = len(clauses)
    Q = [[0] * (n + 1) for _ in range(m + 1)]
    for i, clause in enumerate(clauses, start=1):
        for var in clause:
            Q[i][var - 1] = 1
    return Q

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        m = random.randint(1, n // 2)
        variables, clauses = tseitin_formula(n, m)
        Q = quasi_monogenic_sequence(variables, clauses)
        rank_Q = rank(Q)

        if m < n ** (1/3):
            if rank_Q > 1:
                return {
                    "metric_name": "rank",
                    "metric_value": rank_Q,
                    "instances_tested": 1,
                    "conjecture_holds": False,
                    "counterexample": f"n={n}, m={m}, rank(Q)={rank_Q}"
                }
            else:
                if rank_Q < math.ceil(2 * math.log2(n + m)):
                    return {
                        "metric_name": "rank",
                        "metric_value": rank_Q,
                        "instances_tested": 1,
                        "conjecture_holds": False,
                        "counterexample": f"n={n}, m={m}, rank(Q)={rank_Q}"
                    }

        results.append(rank_Q)

    mean = sum(results) / len(results)
    std = math.sqrt(sum((x - mean) ** 2 for x in results) / len(results))
    support_fraction = all(x >= math.ceil(2 * math.log2(n + m)) for n, m, x in zip(n_values, [random.randint(1, n // 2) for _ in n_values], results))

    return {
        "metric_name": "rank",

```

```

        "metric_value": mean,
        "instances_tested": len(results),
        "conjecture_holds": support_fraction,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2**i + 3 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result["metric_value"])

    mean = sum(results) / len(results)
    std = math.sqrt(sum((x - mean) ** 2 for x in results) / len(results))
    support_fraction = all("conjecture_holds": True for result in results if result["conjecture_holds"])

    if support_fraction:
        print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction=1.0")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b61e7f8f.py", line 119, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b61e7f8f.py", line 88, in run_trial
    Q = quasi_monogenic_sequence(variables, clauses)
        ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b61e7f8f.py", line 70, in quasi_monogenic_sequence
    for var in clause:
TypeError: 'int' object is not iterable

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from verifying the conjecture’s support or falsification. | next: Investigate and fix the error in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12111
2	preregistration	ollama_remote	glm4:latest	0	0	5841
3	novelty	ollama_remote	glm4:latest	0	0	4615
4	novelty	ollama_remote	glm4:latest	0	0	5773
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16783
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10508
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12918
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11478
9	judge	ollama_remote	glm4:latest	0	0	9597

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 89625 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/5178f9916795.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/5178f9916795.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/5178f9916795.tar.gz (if generated)